

ESOF 322

Software Engineering

LECTURE: 2

J. L. Pach

OUTLINE:

- History of GIT
- GIT
- Review of Navigating the Command Line and Bash
- Git Settings
- GIT commands

HISTORY

of GIT

1. CVS – CONCURRENT VERSIONS SYSTEM (C. 1990–2008)

One of the first widely used version control systems.

Characteristics:

- Managed versions of individual files, not entire projects.
- Centralized model – a single main repository.

Limitations:

- Poor branching support.
- No strong data integrity.
- Slow operations.
- Used in early open-source projects, including Linux in its early stages.

2. BITKEEPER (2000–2018)

A **commercial**, distributed version control system known for speed.
From 2002, it was **free for the Linux community** under a special license.

Why it mattered:

- Linus Torvalds used it to manage the Linux kernel codebase.

Crisis in 2005:

- Andrew Tridgell created a tool called SourcePuller by reverse-engineering BitKeeper's protocol.
- BitMover (Larry McVoy's company) considered this a license violation and revoked free access.

2. BITKEEPER (2000–2018)

What happened next:

- In 2016, BitKeeper was open-sourced under the Apache 2.0 license.
- Last release: 2018.
- Today, it’s practically abandoned—Git completely replaced it.

3. THE BIRTH OF GIT (2005)

After losing BitKeeper, Linus Torvalds set requirements for a new tool:

- **Free and open source.**
- **Distributed** (every user has the full history).
- **Extremely fast** (apply a patch in <3 seconds).
- **Resilient to corruption.**

Timeline:

- April 3, 2005 – development begins.
- June 2005 – first release.

3. THE BIRTH OF GIT (2005)

The name ***Git***:

- *Global Information Tracker* (when it works well).
- *Goddamn Idiotic Truckload of ***__ (when it doesn't).

  Officially: *the stupid content tracker.*

GIT - GLOBAL INFORMATION TRACKER

- Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.
- Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

Docs: https://git-scm.com/doc

WHAT IS GIT? (SIMPLE DEFINITION FOR BEGINNERS)

Git is a version control system. Its main job is to track every change in your code and allow you to go back to previous versions if needed. It also lets you compare changes between versions.

But **Git** does more than that:

1. It allows multiple developers to work on the same project at the same time by creating alternative versions of the code, called branches.
2. These branches can later be merged back together. If two people changed the same part of the code, Git will show a conflict that needs to be resolved.
3. This makes Git perfect for collaborative work and for projects that evolve quickly.

GIT IN AGILE & PLAN-DRIVEN

- In Agile development, where code changes often and quality can drop over time, Git helps by making refactoring and merging much easier.
- In more rigid, plan-driven approaches, branching and merging might be less frequent, but Git is still useful for tracking history and avoiding mistakes.

WINDOWS VS LINUX

Command Conventions

WINDOWS VS LINUX COMMAND CONVENTIONS

- In Windows command-line environments such as Command Prompt (cmd) (and the more modern PowerShell), you typically run programs by typing their name, for example:

```
ping
```

- This actually runs ping.exe (the operating system automatically appends the .exe or .com extension). After the command, you add parameters separated by spaces.
- In Windows, **parameters are usually preceded by / (slash) or sometimes - (hyphen/dash).**

WINDOWS VS LINUX COMMAND CONVENTIONS

The traditional way to get help for a command is by using /? or -h or /h. For example:

`ping /?` , `dir /?`

This displays the available options for that command.

```
C:\>ping /?
Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name
Options:
    -t             Ping the specified host until stopped.
                   To see statistics and continue - type Control-Break;
                   To stop - type Control-C.
    -a             Resolve addresses to hostnames.
    -n count       Number of echo requests to send.
```

WINDOWS VS LINUX COMMAND CONVENTIONS

In Linux/Unix systems, a different convention is used:

- Parameters are preceded by - **(single dash)** for short options (usually one letter), and these can often be combined.
- Parameters are preceded by -- (**double dash)** for long, descriptive options, which cannot be combined.

```
student@pi5v:~ $ ping --help
Usage
  ping [options] <destination>
Options:
  <destination>      dns name or ip address
  -a                 use audible ping
  -A                 use adaptive ping
  -B                 sticky source address
  -c <count>         stop after <count> replies
```

WHY GIT WORKS

on Multiple Consoles in Windows

Since Git was originally developed to manage the Linux kernel and is open source, the Windows version of Git can run in at least three different command-line environments:

- **Git Bash**: A Linux-like shell that recognizes common Linux commands such as ls, touch, mkdir, etc.
- **Command Prompt (cmd)**: The classic Windows command line, compatible with MS-DOS conventions.
- **PowerShell**: A powerful Windows shell, significantly different from cmd. It supports some Linux-like commands, but there are important differences between PowerShell and Bash, which often confuse beginners when using Git in PowerShell.

WHY GIT WORKS

on Multiple Consoles in Windows

Regardless of which console you use, Git follows the Linux/Unix convention for command-line options:

- A single dash (-) for short, single-letter options (which can often be combined).
- A double dash (--) for long, descriptive options (which cannot be combined).

Console	Typical Use	Linux Commands Supported?
`Git Bash`	Yes	Fully supported
`CMD`	Yes	No
`PowerShell`	Yes	Partially (aliases)

GIT SETTINGS

Let’s start by configuring Git. Every user—whether on Windows or Linux—has their own local Git settings. Here are the key commands:

```
git config --global user.name "Jacob Pach"       # Sets your name for commits
git config --global user.email "jpach@mtech.edu" # Sets your email for commits
git config --global core.editor "code --wait"    # Sets VS Code as the default editor
git config --global -e                           # Opens the global config file for editing
git config --global core.autocrlf true           # Handles line endings (important on Windows)
```

GIT CONFIG

--global core.autocrlf true

This Git configuration command ensures consistent handling of line endings across different operating systems. When set to true, Git automatically converts Windows-style carriage return + line feed (CRLF) to Unix-style line feed (LF) when committing, and converts back to CRLF when checking out files on Windows. This helps prevent issues caused by inconsistent line endings in collaborative projects involving multiple platforms.

CRLF <--> LF

GIT COMMANDS

- init
- status
- add
- commit
- log

GIT INIT

This command initializes a new Git repository in the current directory. It creates a hidden .git folder that stores all version control data. After running git init, you can start tracking changes, committing files, and using other Git features. It's typically the first step when starting a new project with Git.

```
.git
+---hooks
+---info
+---logs
+---objects
+---refs
|   COMMIT_EDITMSG
|   config
|   description
|   HEAD
|   index
```

GIT STATUS

This command displays the current state of the working directory and staging area. It shows which files have been modified, staged for commit, or are untracked. It helps developers understand what changes are pending and whether they need to add, commit, or discard changes. It's a key tool for tracking progress and avoiding mistakes during version control.

```
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

GIT `STATUS -S` - SHORT STATUS VIEW

This command shows a condensed version of git status, making it easier to quickly scan changes. It uses symbols to indicate the state of each file:

- ?? – Untracked file (not added to Git yet)
- A – File added to staging area
- M – Modified file
- D – Deleted file
- R – Renamed file

Each line shows the status and the filename, making it ideal for fast reviews during development. `git status -s` is especially useful when working with many files. It helps you stay focused and avoid clutter from long status messages.

GIT WORKFLOW: UNDERSTANDING THE THREE MAIN AREAS

- **Directory / Working Space**

  This is where you actively edit files. Changes made here are not yet tracked by Git unless added to the staging area. It reflects your current work-in-progress.

- **Staging Area**

  Also known as the index (“checkpoint”), this is a preparation zone where you place changes that you intend to commit. It allows you to selectively group changes before saving them to the repository.

- **Repository**

  This is the permanent storage area where committed changes are saved. It contains the full history of your project and allows you to track, revert, or collaborate on code over time.

ADDING A FILE TO OUR DIRECTORY

Let's try adding a file to our directory by creating the first file, `firstFile.txt`, using the touch command. Then, we can check the status of our repository to see how Git tracks this new file.

```
$ touch firstFile.txt
$ git status
On branch master
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        firstFile.txt
nothing added to commit but untracked files present (use "git add" to track)
```

ADDING A FILE TO OUR DIRECTORY

```
$ touch firstFile.txt
$ git status
On branch master
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        firstFile.txt
nothing added to commit but untracked files present (use "git add" to track)
```

Git has detected a new file, `firsFile.txt`, but it is untracked, meaning it is not yet being monitored by Git, and suggests using `git add <file>` to start tracking the file and include it in the next commit. This is a typical state right after creating a new file in a freshly initialized repository.

GIT ADD – WILDCARDS AND PATTERNS

Git allows you to use patterns to add multiple files at once. Here's what each one does:

- `git add *.*` - Adds all files in the current directory that have a dot in their name - typically files with extensions (e.g., .txt, .js, .py). It won't add files without extensions.
- `git add *.txt` - Adds only files with the .txt extension in the current directory. This is useful when you want to stage a specific type of file.
- `git add .` - Adds all changes in the current directory and subdirectories - including new, modified, and deleted files. This is the most comprehensive option and is commonly used when you're ready to stage everything.

GIT ADD – BE CAREFUL WITH BULK ACTIONS

Although Git offers graphical interfaces and many conveniences — including the ability to add multiple files at once using patterns like git add . — in practice, especially early in your Git journey (and even later as a professional), it's often safer to add files individually or in a carefully reviewed list.

Why?

Bulk adding (`git add .` or `git add *.txt`) can easily include unintended changes.

Mistakes in staging can lead to confusing commits or even bugs. While it's possible to revert to a previous commit, it’s more complex than simply taking a moment to review and add files intentionally.

FIRST COMMIT – GIT COMMIT

- When you run git commit, Git opens your default text editor (like Notepad or VSC) and waits for you to enter a commit message. This message should briefly describe the changes you're saving to the repository.
- The message is important because it becomes part of the project's history and helps others (and your future self) understand what was changed and why.

```
$ git commit
hint: Waiting for your editor to close the file...
```

COMMIT MESSAGE CONVENTIONS

It’s important to be consistent in how you write commit messages. Two common styles are:

- **Present tense** (recommended): Fix bug in login form. Add validation to email field. This style is preferred because it describes what the commit does to the codebase.
- **Past tense**: Fixed bug in login form. Added validation to email field. This is also acceptable, but less common in collaborative projects.
- **Avoid progressive tense** (e.g., “fixing”, “adding”) because it suggests an ongoing action, not a completed change.

Use short, clear messages in the present tense to describe what your commit does. This keeps the project history clean and easy to read. Choose one style and be consistent.

GIT COMMIT – EDITOR MESSAGE FLOW

- When you run git commit without the `-m` flag,
  - Git opens your default text editor
  - in our case, Visual Studio Code (VSC)
  - and waits for you to write a commit message.

```
Add firstFile.txt
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
#
# Initial commit
#
# Changes to be committed:
#       new file:   firstFile.txt
#
```

GIT COMMIT – EDITOR MESSAGE FLOW

```
Add firstFile.txt
# ...
#
```

- The first line of the message should be short and descriptive
  - this is the summary that appears in logs and history.
- You can optionally add a longer description below, separated by a blank line, to explain the change in more detail.

The rest of the file may contain comments or instructions from Git (lines starting with #). These can be left as-is or removed. Once you're done, save and close the file. Git will then finalize the commit and return you to the terminal.

USING GIT COMMIT -M "MESSAGE"

Shortcut for Commit Messages

Instead of typing git commit and opening the default editor, you can write your commit message directly in the terminal using: `$ git commit -m "Your commit message"`

- This saves time and avoids switching to the editor. The message should be short and descriptive.
- Use `-m` when your message is simple and clear. For longer or multi-line messages, it's better to use the editor to provide more context

GIT LS-FILES – LIST TRACKED FILES

This command displays all the files that Git is currently tracking in your repository. It shows the contents of the index (staging area), not the working directory. That means:

- Files listed by git ls-files are already added to Git using git add.
- Untracked files (new files not yet added) will not appear in this list.
- It's useful for checking which files are under version control, especially in large projects.

```
$ git ls-files
firstFile.txt
```

Use git ls-files to verify which files are being tracked by Git. If a file doesn't appear, it's either untracked or ignored via .gitignore.

GIT LOG – COMMIT HISTORY

This command displays the complete commit history of the repository. Each entry includes:

- The commit hash (a unique ID)
- Author name and email
- Date and time of the commit
- The full commit message
- It's useful for reviewing detailed information about each change made to the project.

```
$ git status
On branch master
nothing to commit, working tree clean
```

GIT LOG – EXAMPLE

This command displays the complete commit history of the repository

```
$ git log
commit d06aafaf2cd37f5bc7cd4015656e1ae15241c996 (HEAD -> master)
Author: Jacob Pach <jpach@mtech.edu>
Date:   Thu Aug 28 20:09:17 2025 -0600

    Add firstFile.txt
```

GIT LOG --ONELINE – SIMPLIFIED VIEW

This version shows a condensed list of commits, with:

- A shortened commit hash
- The first line of the commit message

It's ideal for quickly scanning the history or identifying specific commits without all the extra details.

```
$ git log --oneline
d06aafa (HEAD -> master) Add firstFile.txt
```

Use git log when you need full context, and git log --oneline when you want a quick overview. Both are essential tools for navigating and understanding your project's history.

REMOVING COMMITTED FILES IN GIT

Important Note

Be careful when removing files that have already been committed. If, for example, firstFile.txt is no longer needed in future commits, and you delete it manually from the repository folder, Git won't automatically recognize this change.

To inform Git that the file was intentionally removed, you must add the deletion using:

```
git add firstFile.txt
```

- This may seem counterintuitive, but it tells Git: *I want this deletion to be part of the next commit.* Once committed, the file will no longer exist in the current branch.

REMOVING COMMITTED FILES IN GIT

Important Note

Of course, you can always restore the file by checking out a previous commit where it still existed.

Git tracks changes — including deletions — only when you explicitly stage them.

Use git add even for removed files to make the change part of your commit history.

RENAMING FILES IN GIT

Important Note

When working with Git, it's important to understand how file renaming is handled. Git does not automatically detect a rename as a single action. Instead, it treats it as:

- Deletion of the old file
- Creation of a new, untracked file

So, if you rename a file manually (e.g., from `oldName.txt` to `newName.txt`), Git will see `oldName.txt` as deleted and `newName.txt` as a new file. To properly reflect this change in Git, you should:

```
git add oldName.txt
git add newName.txt
git commit -m "Renamed file from oldName.txt to newName.txt"
```

SHORTCUT - GIT COMMIT -AM "MESSAGE"

Stage and Commit in a Single Command

The `-am` flag combines staging (**a**dd) and committing (**m**essage) into one step:

```
git commit -am "Your commit message here"
```

- **Saves Time**: Eliminates the separate `git add .` step for modified files.
- **Keeps Focus**: Keeps your workflow fast when making quick, iterative changes.
- **Cleaner Terminal History**: Reduces repetitive status checks and staging commands.

Important Note: This flag only works for modified or deleted tracked files. Brand new (**untracked**) files still require an explicit git add `<file>`.

THANK

You